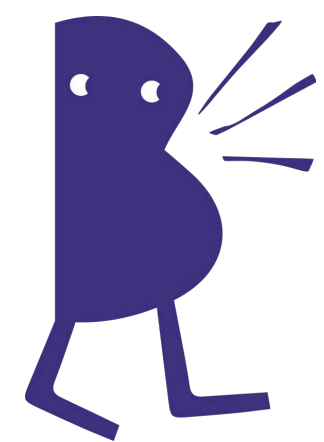


Correctness Too Cheap To Meter

Formal Verification and LLMs

Emilie Ma, UBC; in collaboration with Qian Cheng, NJU; Ruize Tang, MSR Asia;
Finn Hackett, UBC; Ivan Beschastnikh, UBC; Yu Huang, NJU; Tianyin Xu, UIUC

<https://emilie.ma/bb2026>



Motivation



Formal methods protect critical systems.

TLA+ verifies Intel chips, Azure DBs, spacecraft, and more [1].

But they're expensive and time-consuming.

It takes months to years of expert effort to specify a system.

Can AI help automate developer toil?

Are LLMs able to generate system specs, and are these specs *useful*?

Contents



What are formal methods?



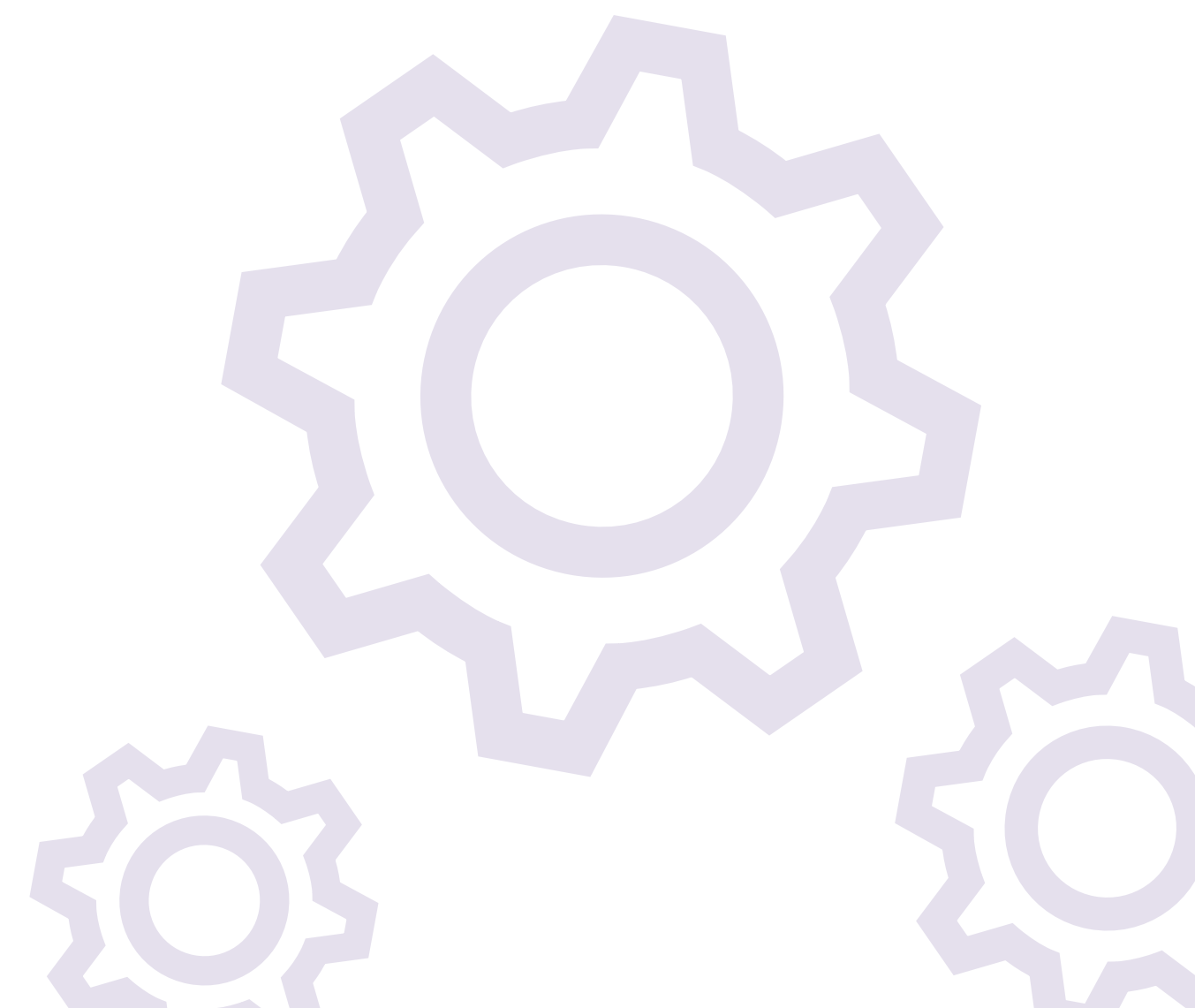
Can LLMs generate correct, useful specs?



Can agents use specs to find bugs?



What does this mean for formal methods?



TLA⁺

- De-facto modelling language for distributed and concurrent systems
- Used by Google/AWS/MSR
- Write abstract spec of your code, specify some invariants

lock.py

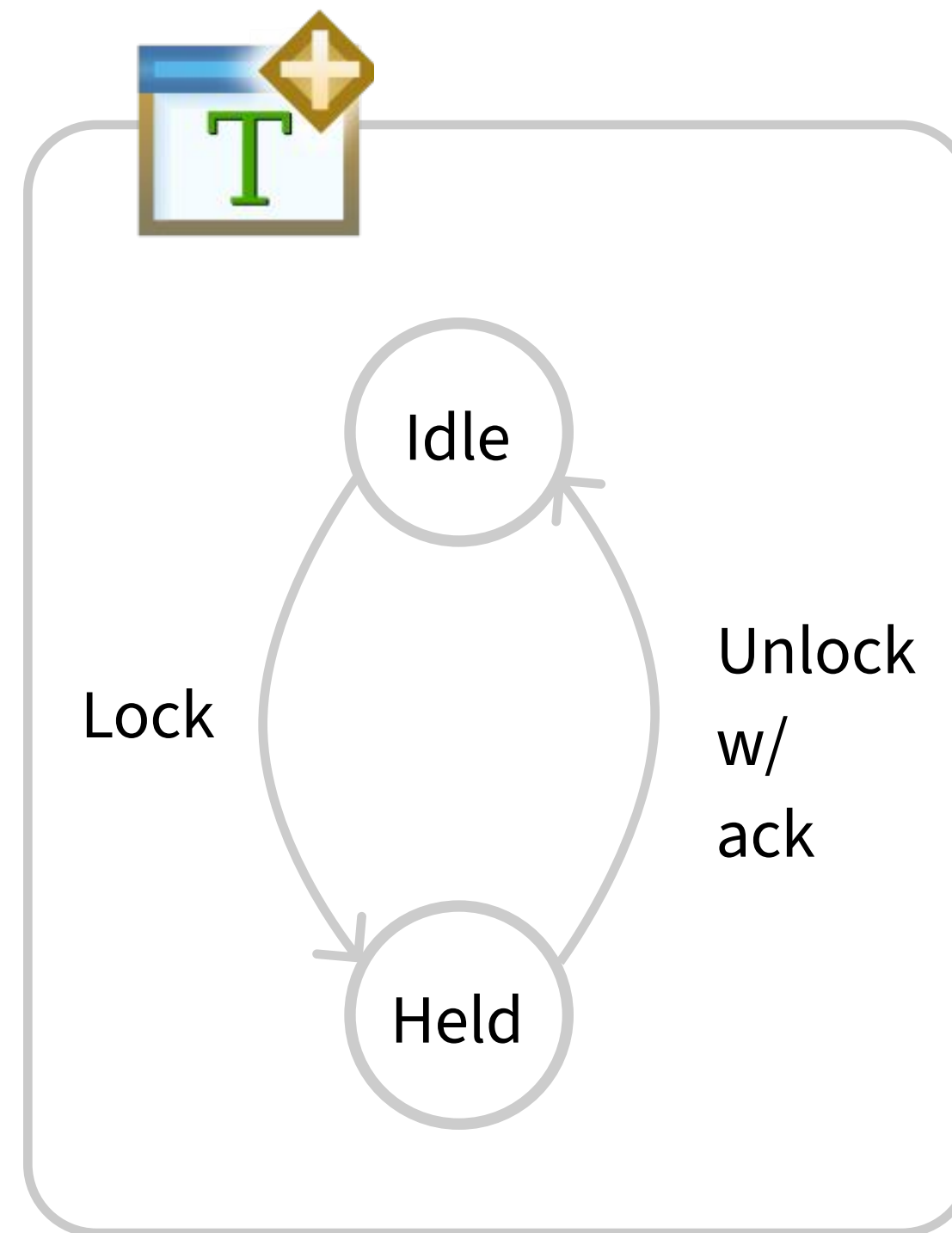
```
1 state = "idle"
2 ack = False
3
4 def handle(cmd):
5     global state
6     if cmd == "lock":
7         if state == "idle":
8             state = "held"
9             return "ok"
10
11     if cmd == "unlock":
12         if state == "held":
13             if ack:
14                 state = "idle"
15                 return "ok"
16     # etc
```

lock.tla

```
1 Init ==
2   /\ state = "idle"
3   /\ ack = FALSE
4 Lock ==
5   /\ state = "idle"
6   /\ state' = "held"
7   /\ ack' = ack
8 Unlock ==
9   /\ state = "held"
10  /\ ack
11  /\ state' = "idle"
12  /\ ack' = ack
13 Next == Lock \/ Unlock
14 TypeOK ==
15   /\ state \in {"idle", "held"}
16   /\ ack \in BOOLEAN
17 NoDeadlock == ENABLED Next
```

TLC

- Model checker for TLA+
- Exhaustively explores all paths in spec
- Checks invariants at each step → any violations mean bug in spec or code
- Guards against hallucinations



✗ NoDeadlock Violated

lock.tla

```
1 Init ==
2   /\ state = "idle"
3   /\ ack = FALSE
4 Lock ==
5   /\ state = "idle"
6   /\ state' = "held"
7   /\ ack' = ack
8 Unlock ==
9   /\ state = "held"
10  /\ ack
11  /\ state' = "idle"
12  /\ ack' = ack
13 Next == Lock \/ Unlock
14 TypeOK ==
15   /\ state \in {"idle", "held"}
16   /\ ack \in BOOLEAN
17 NoDeadlock == ENABLED Next
```

Trace Validation

- Ensures spec conforms with implementation
- Get execution traces from system, check if these logs have corresponding paths in the spec
- Requires instrumenting system (by hand)
- Also guards against hallucinations

lock.log

```
00:01 Lock
00:02 Unlock
00:03 Lock
```

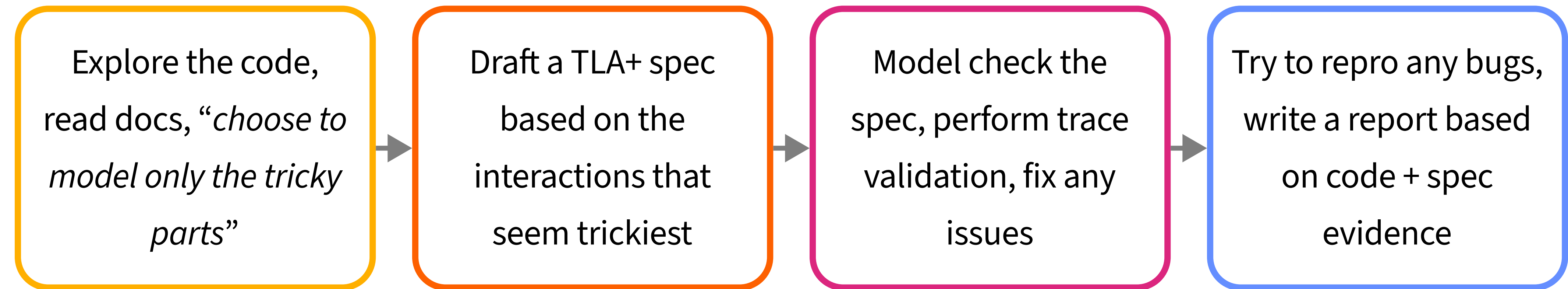
✗ **Unlock → Lock in log,
not in model**

lock.tla

```
1 Init ==
2   /\ state = "idle"
3   /\ ack = FALSE
4 Lock ==
5   /\ state = "idle"
6   /\ state' = "held"
7   /\ ack' = ack
8 Unlock ==
9   /\ state = "held"
10  /\ ack
11  /\ state' = "idle"
12  /\ ack' = ack
13 Next == Lock \/ Unlock
14 TypeOK ==
15   /\ state \in {"idle", "held"}
16   /\ ack \in BOOLEAN
17 NoDeadlock == ENABLED Next
```

Typical TLA⁺ Workflow

Let's say a developer wants to formally verify a new distributed system.



Sounds like a lot of work...

TLA+ is powerful but costly.

The time/effort required to write specs limits its adoption in practice.

Specs separated from code.

Specs may quickly diverge from implementations, and updating them causes toil.

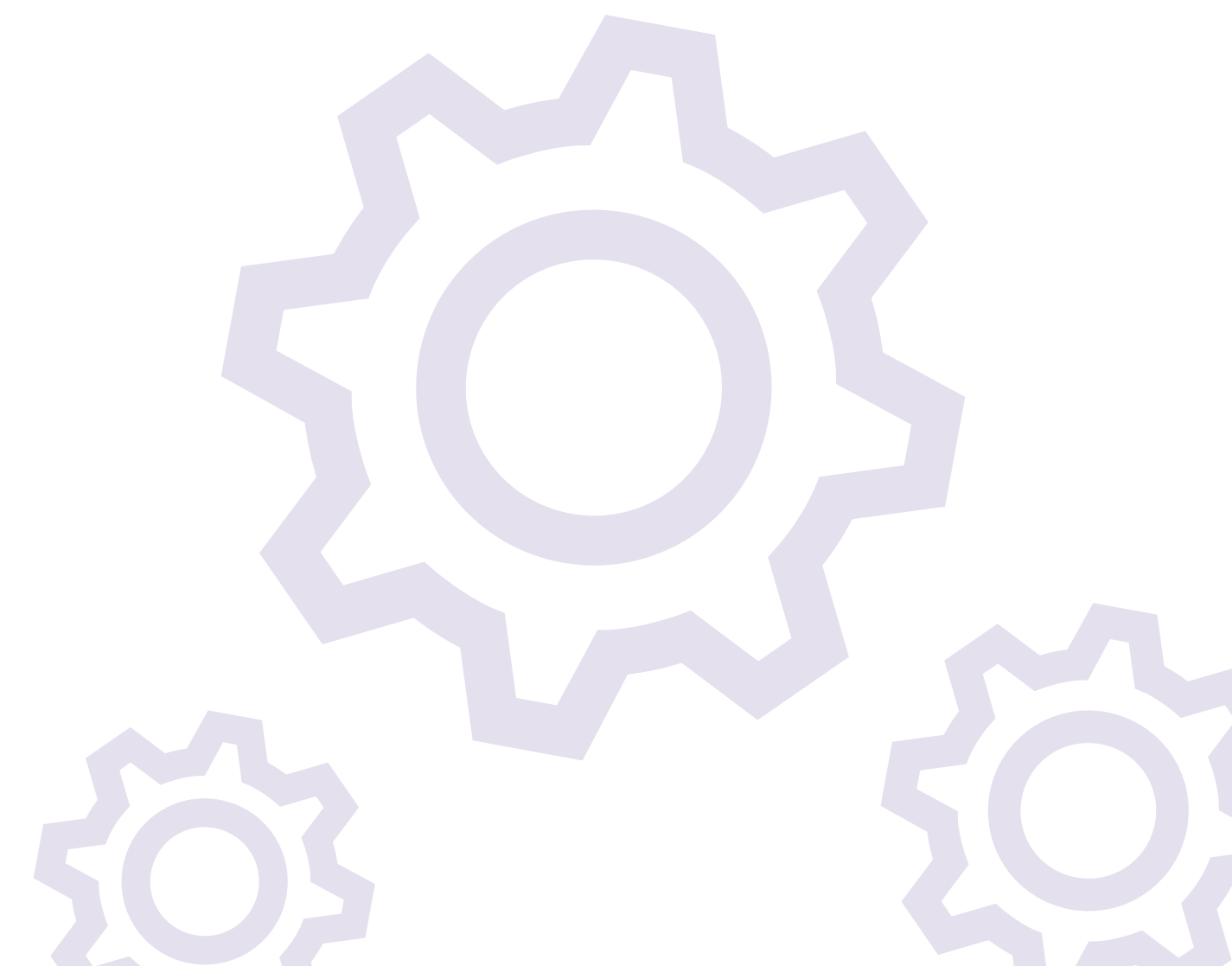
Nothing can help.

No prior work exists to reliably automate any of this workflow in general.



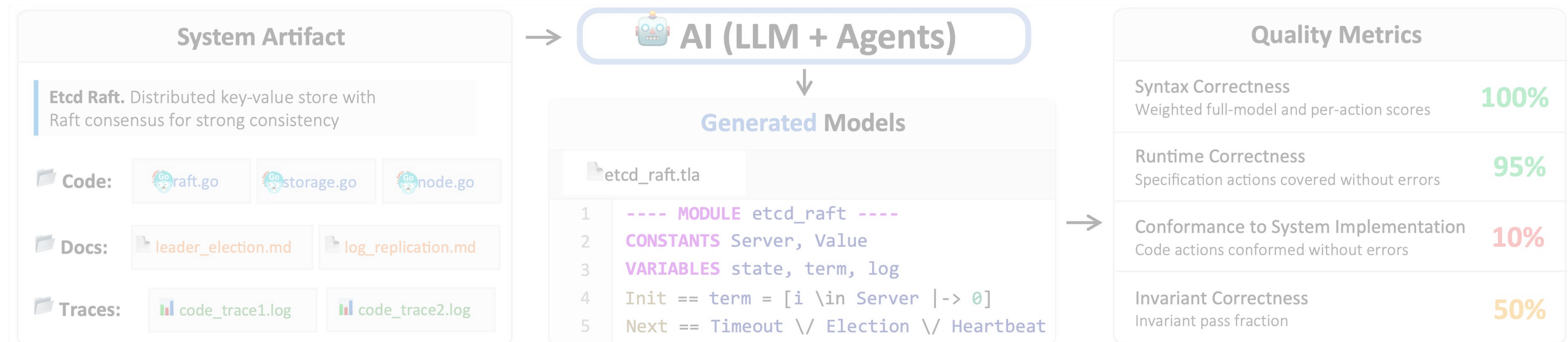
What's a good spec?

- Is it syntactically valid? – this is the extent of existing benchmarks (Cao et al., 2025; TLAi+Bench, 2025)
- Is the spec self-consistent?
- Does the spec match actual system execution traces?
- Does the spec reflect basic properties of the system? (e.g., no deadlock)



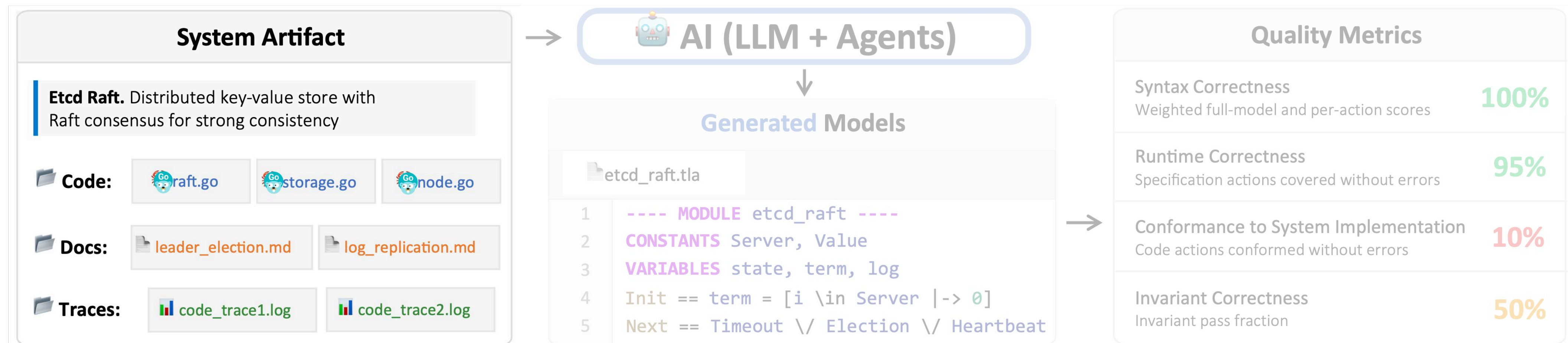
★ SysMoBench

- Benchmark of LLM capabilities to generate useful specs of real-world systems
- Automated metrics: syntax validity, runtime correctness, conformance, invariant correctness
- Across 11 artifacts, 3 agents, 3 modelling languages (primarily TLA+)



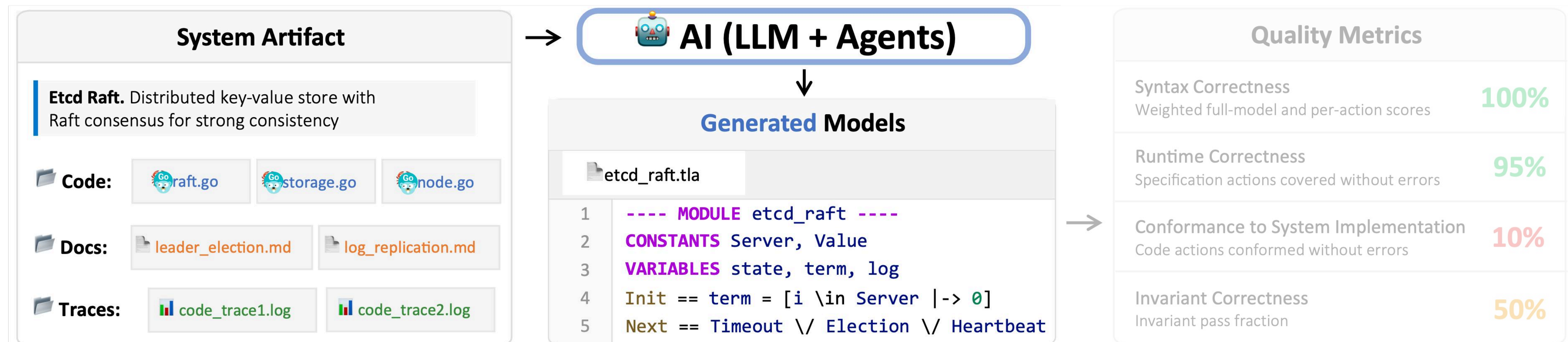
★ SysMoBench

- Benchmark of LLM capabilities to generate useful specs of real-world systems
- Automated metrics: syntax validity, runtime correctness, conformance, invariant correctness
- Across 11 artifacts, 3 agents, 3 modelling languages (primarily TLA+)



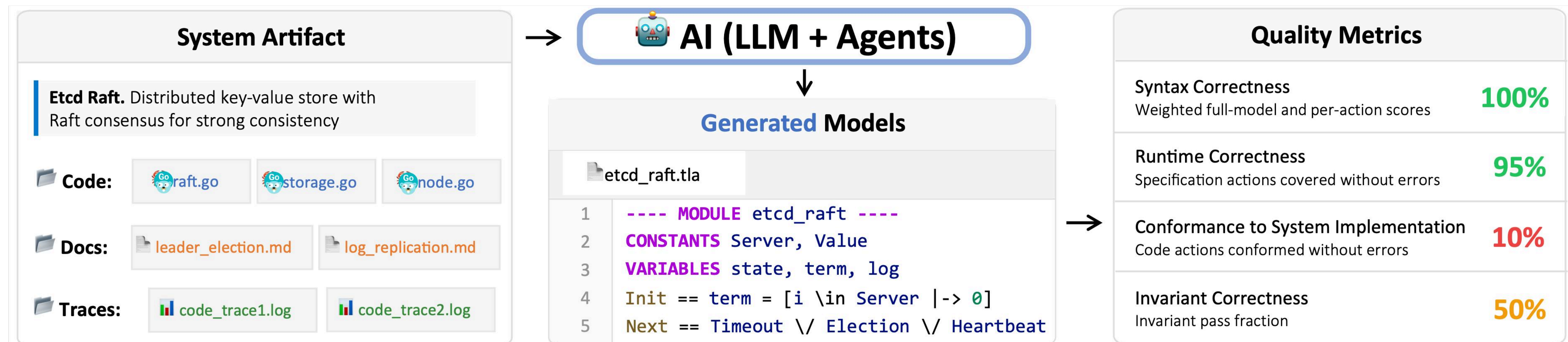
★ SysMoBench

- Benchmark of LLM capabilities to generate useful specs of real-world systems
- Automated metrics: syntax validity, runtime correctness, conformance, invariant correctness
- Across 11 artifacts, 3 agents, 3 modelling languages (primarily TLA+)



★ SysMoBench

- Benchmark of LLM capabilities to generate useful specs of real-world systems
- Automated metrics: syntax validity, runtime correctness, conformance, invariant correctness
- Across 11 artifacts, 3 agents, 3 modelling languages (primarily TLA+)



Results

- High syntax scores → wealth of TLA+ examples
- Poor conformance/invariant scores, demonstrating LLMs still incapable of sufficient system reasoning
- Consistent trends across LLMs

Can LLMs generate correct, useful specs?
Not quite, but getting there!

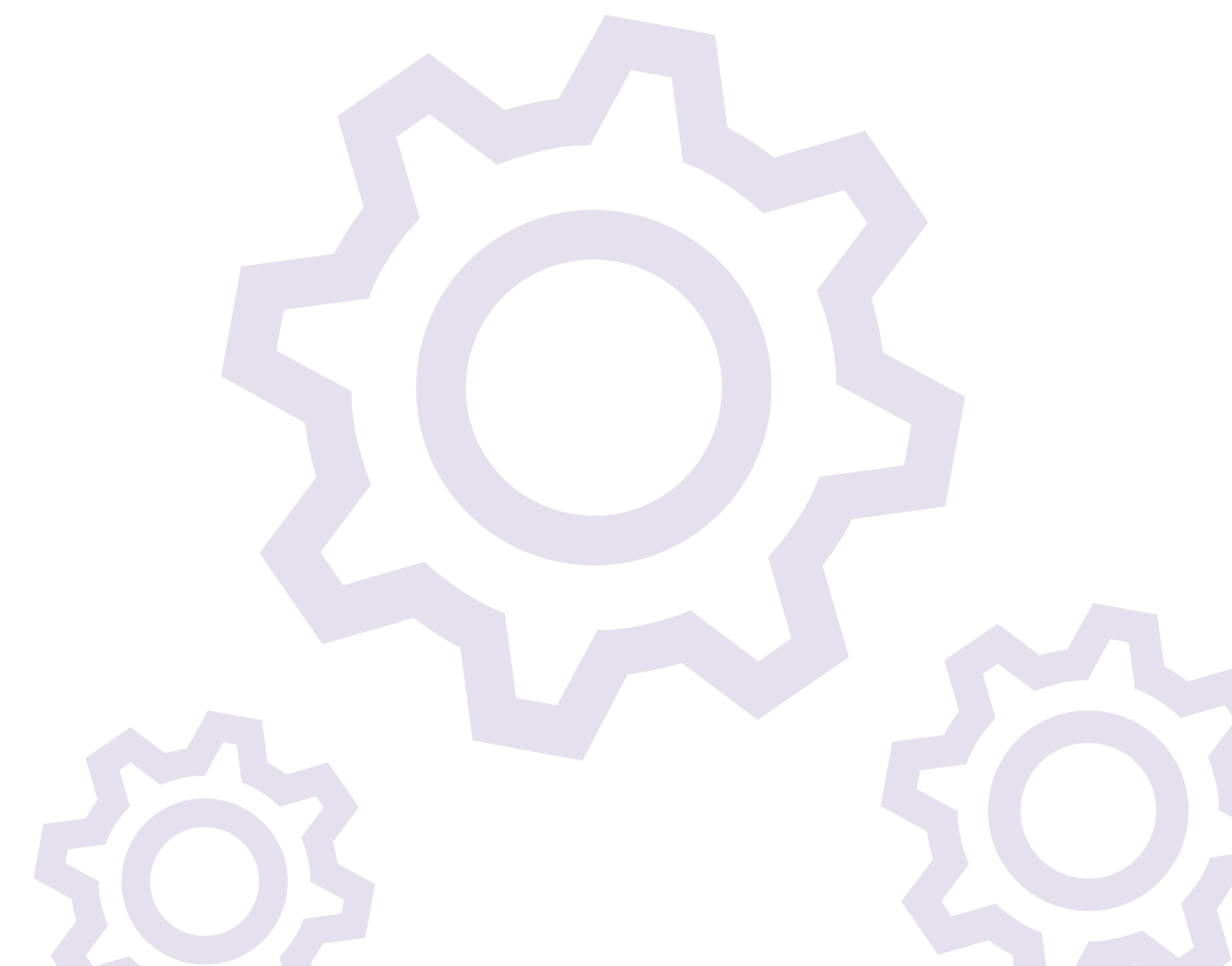
#	MODEL	SYNTAX	RUNTIME	CONFORMANCE	INVARIANT	SCORE ↓
1	 Gemini 3.1 Pro	100.0	92.4	84.2	80.6	89.3
2	 GPT-5.4	100.0	74.8	63.6	40.3	69.7
3	 Claude Opus 4.7	100.0	66.7	60.0	51.3	69.5
4	 Claude Sonnet 4.6	100.0	77.0	44.2	55.6	69.2
5	 Grok 4	95.5	64.5	50.6	51.0	65.4
6	 Kimi K2.5	100.0	64.8	43.9	32.5	60.3
7	 GPT-5.2	75.6	30.0	27.3	25.1	39.5
8	 DeepSeek V3.2	70.2	36.4	24.2	22.1	38.2

sysmobench.com



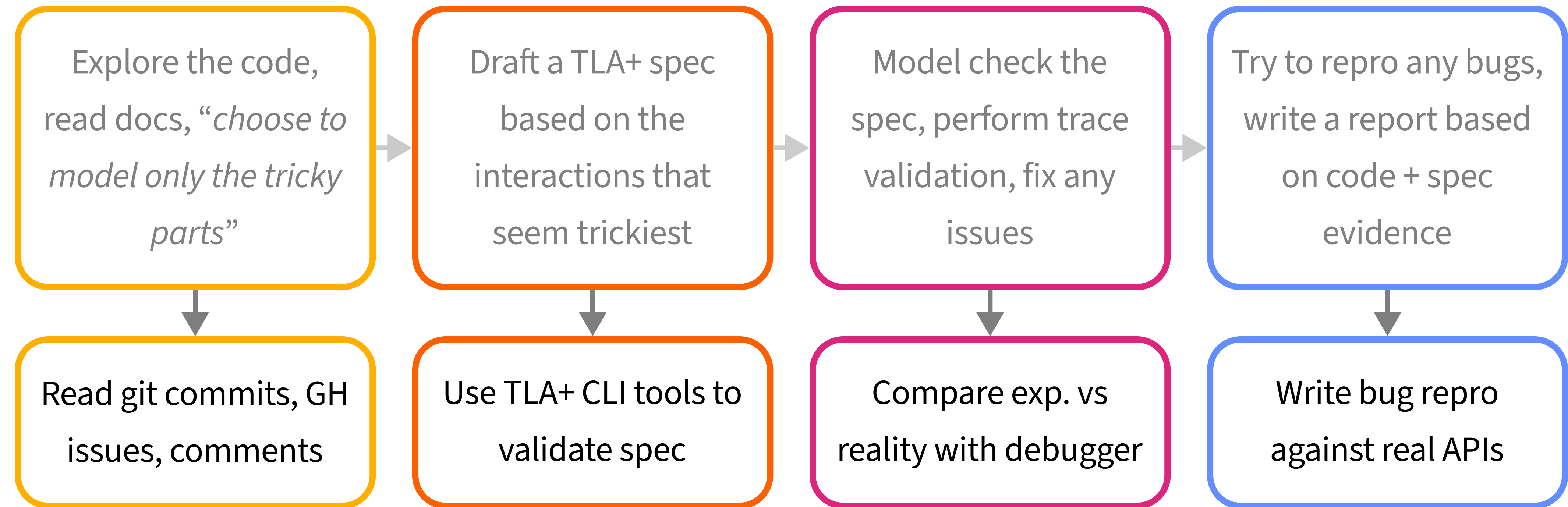
Challenge: Conformance

- Can't use the specs we generate if they don't match the system
- Specs must be precise, anchored in the code, match execution traces
- To debug spec errors, LLMs need access to some “reality” to compare against
- Bugs only matter if they can be triggered and observed in practice
- What abilities do LLMs need to realize the above?



Revisiting the workflow.

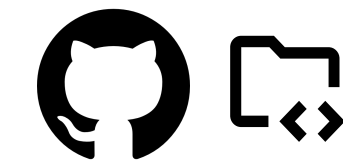
The abilities LLMs need:



★ Specula

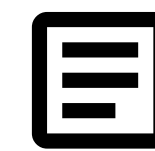
- Four-phase agentic pipeline for spec generation and bug discovery
- Built on top of coding agent w/ Agent Skills + MCPs
→ improves SysMoBench results to **100%**
- **165** bugs so far across projects like MongoDB, Hashicorp Raft, libgomp
- No human intervention, reasonable cost

Phase 1: Repository Mining



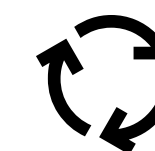
Identify core modules,
project-specific context

Phase 2: Initial Spec Generation



Generate the spec, instrument
the system

Phase 3: Spec Refinement



Iterate between trace
validation and model checking

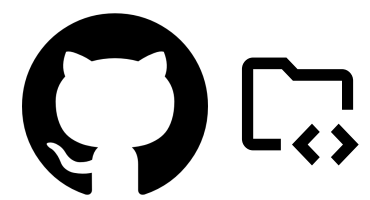
Phase 4: Bug Confirmation



Confirm bugs via dev comments,
generate reproductions

Phase 1

Phase 1: Repository Mining

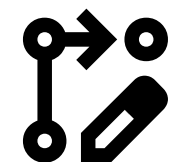


Identify core modules,
build project-specific
context

Data sources:



Issues/PR
discussion
on GitHub



Git commit
history



Individual
files (impl.
details)

Outputs



modelling_brief.md

Summarize suspicious
findings to focus modelling

- Overview of development history to identify suspicious modules, infers invariants and how fine-grained to make spec
- Takes in codebase as input

Phase 2

Phase 2: Initial Spec Generation



Use the `tla2sany` and `spec_analyzer` MCPs

Outputs



① **initial_spec.tla**
Main impl. level system spec



② **MC.tla**
Bounds for perf., more invs.



③ **Trace.tla**
Maps traces to spec actions



instrumentation.md
Describes code changes
necessary to log events

- Generates spec based on Phase 1 modelling brief (three variants sharing same base)
- Instruments code in preparation for trace validation

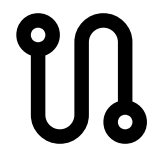
Phase 3

Phase 3: Spec Refinement



Use the trace_debugger and MC_summary MCPs

Iterative process:



Trace
validation:
spec $\leftrightarrow$ impl



Repeat
until spec
converges



Model
checking: no
bad states

Outputs



Corrected specs

Files will be updated in-place during iteration

- Iterates between trace validation and model checking to ensure spec conforms
- Uses custom step-through debugging and state diffing tools, like a human

Phase 4

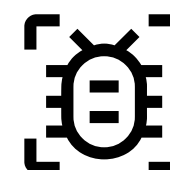
Phase 4: Bug Confirmation



After spec converges, rerun model checking to find bugs



Match developer intent: read issues, comments to determine if behaviour intended



Reproduce bugs: write test to trigger bug in real usage

Outputs



Bug reports

Repro steps, TLA+ spec evidence; ready to file

- Reads codebase to see if a “bug” might actually be intended behaviour
- Generates a repro scenario to trigger the bug
- Creates a final bug report, collecting code + trace validation + model checking evidence
- Reports ready for manual review: for eval we checked every report before filing issues

Case Study

- libgomp: GCC's parallel programming library
- Little documentation, low-level code
- Focused on a patch submitted by an NVIDIA engineer

Phase 1: Repository Mining

1. System Overview

- *System*: libgomp: GCC's OpenMP runtime library
- *Language*: C (~900 LOC of core barrier logic in `config/linux/bar.c + bar.h`)
- *What it implements*: Flat barrier synchronization for `#pragma omp parallel`
- *Concurrency model*: ...
- *Key architectural choice*: ...

Bug Family 1: BAR_HOLDING_SECONDARIES Lifecycle

Bug Family 2: Cancellation + Barrier Flag Cleanup

...

Case Study

Phase 2: Initial Spec Generation

```
(*
 * Primary enters barrier.
 * Models: bar.h:236-259
gomp_barrier_wait_start (for id==0)
 *      bar.h:256-258: if (id == 0)
ret |= BAR_WAS_LAST
 *)
PrimaryEnterBarrier ==
  /\ pc[Primary] = "idle"
  /\ ~prevHolding  \* team.c:34-50:
gomp_release_held_threads must fire
first
  /\ ...
```

Phase 3: Spec Refinement

```
PrimaryHandleTaskLast ==
  /\ ...
  /\ ELSE \* all tasks done
    IF barrierType = BarrierCancel
      THEN ...
    ELSE
      /\ generation' = generation + 1
      /\ waitingForTask' = FALSE
      /\ pc' = [pc EXCEPT ![Primary] = "done"]

BarrierSafety ==
  \A t \in Secs : pc[t] = "done" =>
    \A s \in Secs : threadGen[s] >= threadGen[t]
```

Case Study

Phase 4: Bug Confirmation

```
static unsigned gomp_increment_gen(...) {  
    // strips ALL flags (bits 0-5)  
    unsigned gens = (gen & BAR_BOTH_GENS_MASK);  
    switch (increment) {  
    case BAR_CANCEL_INCR:  
        return BAR_INCREMENT_CANCEL(gens);  
    case BAR_HOLDING_SECONDARIES:  
        // BAR_CANCELLED lost!  
        return gens | BAR_HOLDING_SECONDARIES;  
    // ...  
}
```

✗ BarrierSafety Invariant Violated

BAR_CANCELLED Overwrite in gomp_barrier_handle_tasks

gomp_increment_gen unconditionally strips all flag bits including BAR_CANCELLED. This can lead to an assertion failure in checking builds and a subtle internal state inconsistency otherwise.

Trigger Path

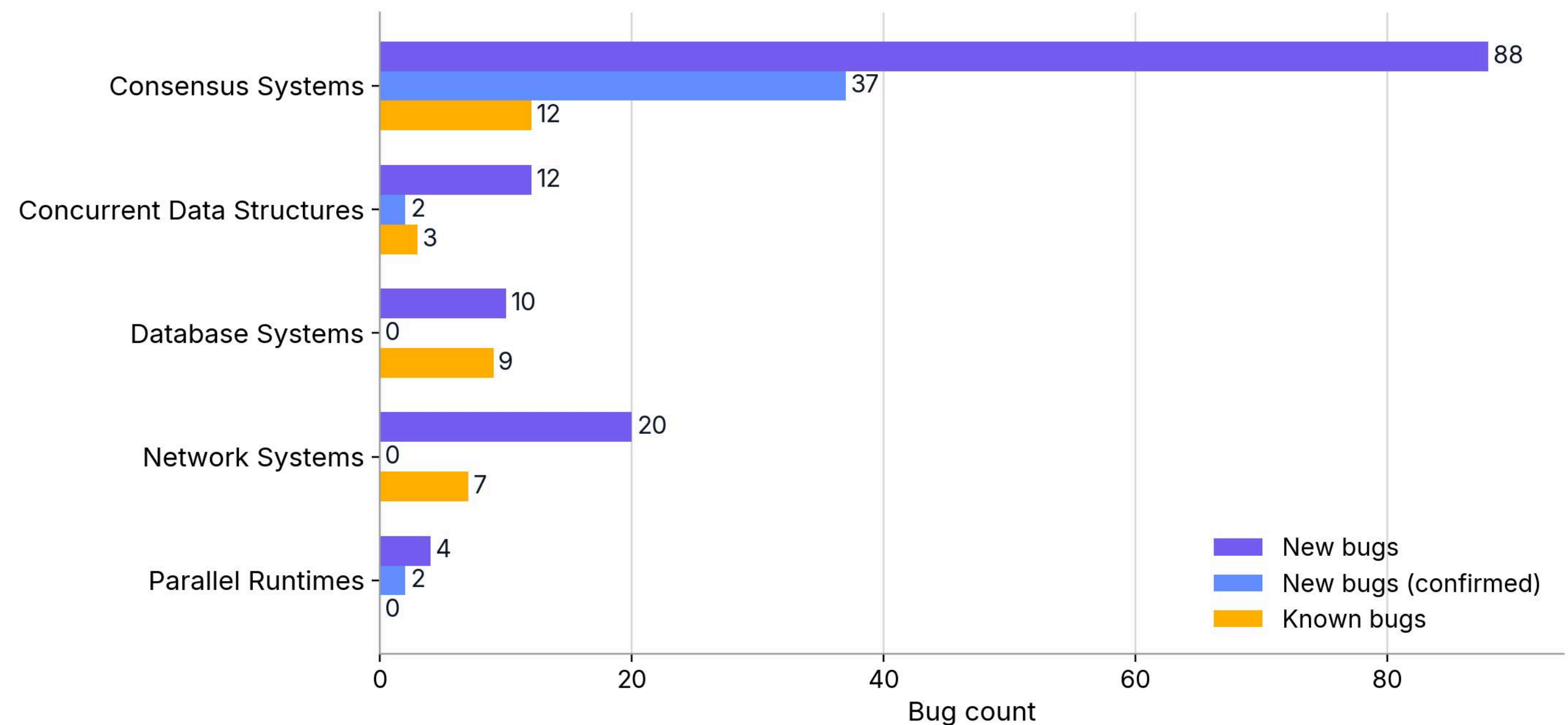
In repro/final_barrier_cancel_task.c ...

Root Cause

...

An impressive number of bugs!

- Total of 165 bugs identified across 47 target repositories
 - **134 previously unknown**
 - 4.7% false positives (2/43)
- Serious bugs: deadlocks, data loss/corruption, memory safety, stale reads
- Specula demonstrates real-world value!



Reasonable token/time cost.

- On average, \$42.40 USD (~36 €) in token costs per target repository
- Avg. 3h10m in runtime
- Cost/runtime mostly spent on repo mining and spec refinement
- ~Affordable for wide-scale application!

Example per-phase cost/time breakdown for libgomp.



Limitations



Repo Mining Heuristics

May miss modules suitable for model-checking if the code has little history.

Conformance Coverage

Model-checked traces cover 85% of code on avg., remainder is unvalidated.

False Bug Confirmations

Some dev assumptions are unstated (e.g., unreasonable config options).



Reflections

- *Can agents use specs to find bugs?* — Yes, very well!
- Demonstrates value and scalability of agents + formal verification, with little human effort
- Complements initiatives like Project Glasswing: formal guarantees with more confidence
- Offers a way to reduce the harms of AI slop/vibecoding



Conclusion

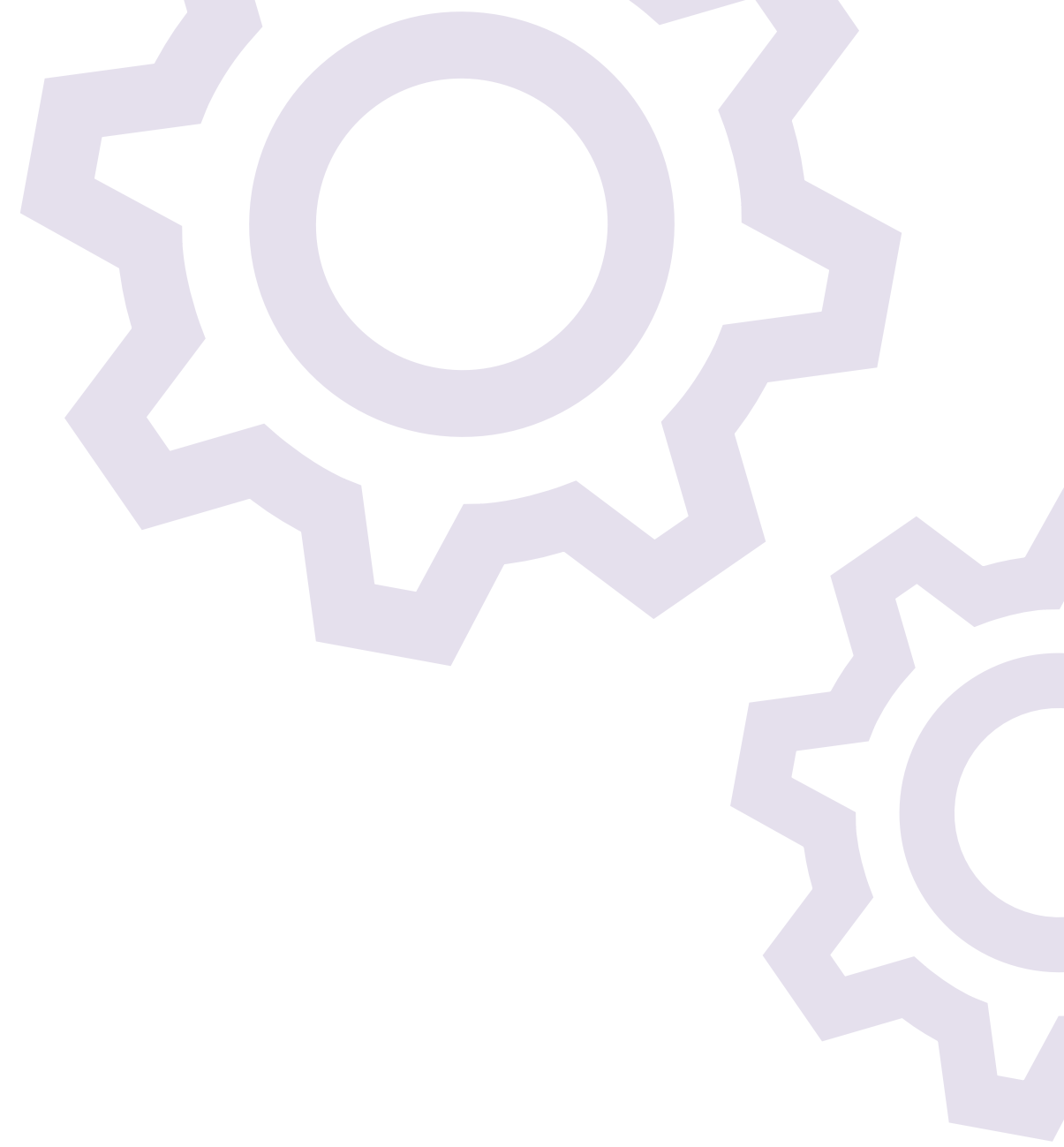
- Agents enable affordable formal methods adoption
- **SysMoBench** quantifies frontier model gaps
 - Raw LLMs still struggle with conformance and invariants
- **Specula** automates spec generation and bug repro
 - Identified 134+ new bugs in production systems
- Vision: verifying all safety-critical systems in the wild!



<https://github.com/specula-org/Specula>



Supplementary Slides



More bugs than baseline agents.

- Compared against other Claude Code configurations (no tools, with official TLA+ MCPs) w/ same prompts
- Controlled experiments on 4 repos, selected for high number of bugs
- Baseline agents unable to identify any bugs; Specula found 9 (out of 10 known)

System	Description	Lang.	LoC
Etcd Raft	Production Raft implementation	Go	6.0K
Autobahn	BFT consensus protocol [10]	Rust	6.5K
libgomp	OpenMP concurrency library	C	5.7K
papaya	Concurrent hash map	Rust	3.9K

Agent	etcd-raft	autobahn	libgomp	papaya	Total
agent-basic	0	0	0	0	0
agent-tla-tools	0	0	0	0	0
Specula	0	5	3	1	9

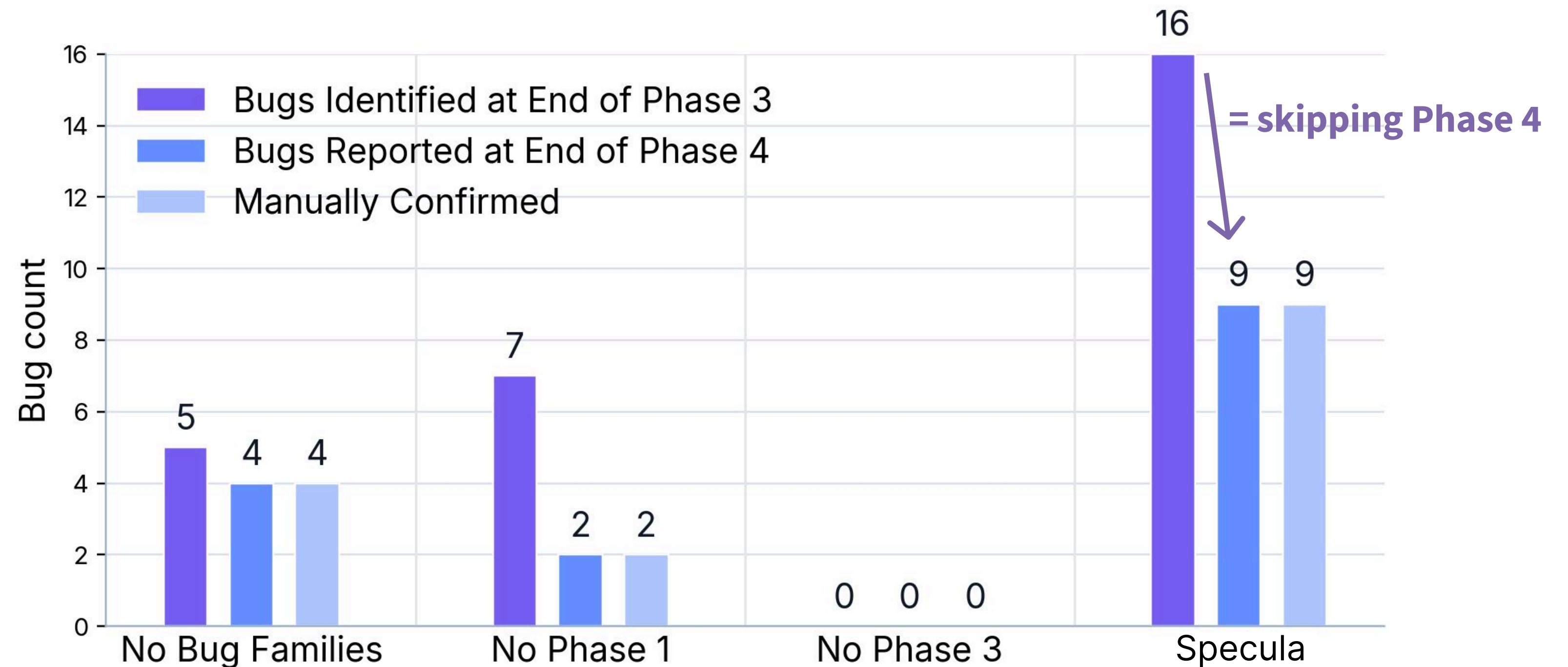
More conformant, quality specs.

- Achieves 100s across all SysMoBench metrics, while baseline agents struggle with conformance
- Specula's extra structure and tooling really helped!

		Syntax Validation	Model Checking	Conformance	Invariant Correctness
etcd-raft	agent-basic	100.00% ✓	100.00% ✓	70.00% ✗	55.56% ✗
	agent-tla-tools	100.00% ✓	100.00% ✓	70.00% ✗	66.67% ✗
	Specula	100.00% ✓	100.00% ✓	100.00% ✓	100.00% ✓
autobahn	agent-basic	100.00% ✓	100.00% ✓	100.00% ✓	0.00% ✗
	agent-tla-tools	100.00% ✓	77.78% ✗	100.00% ✓	37.50% ✗
	Specula	100.00% ✓	100.00% ✓	100.00% ✓	100.00% ✓
libgomp	agent-basic	100.00% ✓	100.00% ✓	57.14% ✗	9.09% ✗
	agent-tla-tools	100.00% ✓	100.00% ✓	57.14% ✗	9.09% ✗
	Specula	100.00% ✓	100.00% ✓	100.00% ✓	100.00% ✓
papaya	agent-basic	100.00% ✓	100.00% ✓	50.00% ✗	5.88% ✗
	agent-tla-tools	100.00% ✓	100.00% ✓	60.00% ✗	35.29% ✗
	Specula	100.00% ✓	100.00% ✓	100.00% ✓	100.00% ✓

Each phase impacts bug finding.

- Phase 1 → number and *types* of bugs found
- Phase 3 → evidence to identify bugs
- Phase 4 → false positives
- Each phase of Specula design is crucial!



SysMoBench Results (Sept.)

- For basic systems (~100 LoC), spec generation looks good
- For large systems (~5K LoC), can't generate syntactically valid specs, nor are they internally consistent
- Even strongest models generate specs that don't actually match system (low Conformance)

(a) Asterinas Spinlock

Agent	LLM	Syntax	Runtime	Conformance	Invariant
Basic Modeling	Claude-Sonnet-4	100.00% ✓	100.00% ✓	100.00%	100.00%
	GPT-5	100.00% ✓	100.00% ✓	80.00%	100.00%
	Gemini-2.5-Pro	100.00% ✓	100.00% ✓	80.00%	85.71%
	DeepSeek-R1	100.00% ✓	100.00% ✓	80.00%	100.00%
Code Translation	Claude-Sonnet-4	100.00% ✓	100.00% ✓	100.00%	100.00%
	GPT-5	100.00% ✓	100.00% ✓	100.00%	85.71%
	Gemini-2.5-Pro	100.00% ✓	100.00% ✓	100.00%	100.00%
	DeepSeek-R1	100.00% ✓	100.00% ✓	100.00%	100.00%

(b) Etcd Raft

Agent	LLM	Syntax	Runtime	Conformance	Invariant
Basic Modeling	Claude-Sonnet-4	100.00% ✓	25.00% ✓	7.69%	69.23%
	GPT-5	47.87% ✗	-	-	-
	Gemini-2.5-Pro	50.00% ✗	-	-	-
	DeepSeek-R1	50.00% ✗	-	-	-
Code Translation	Claude-Sonnet-4	100.00% ✓	66.67% ✓	15.38%	92.31%
	GPT-5	100.00% ✓	20.00% ✗	-	-
	Gemini-2.5-Pro	44.44% ✗	-	-	-
	DeepSeek-R1	100.00% ✓	0.00% ✗	-	-

SysMoBench Takeaways (Sept.)

- LLMs struggled with *basic* specification tasks
- Failed to reason about temporal properties
- Could reproduce old known bugs, couldn't find new ones

Can LLMs generate correct, useful specs?
No, **limited system comprehension**.

(a) Asterinas Spinlock

Agent	LLM	Syntax	Runtime	Conformance	Invariant
Basic Modeling	Claude-Sonnet-4	100.00% ✓	100.00% ✓	100.00%	100.00%
	GPT-5	100.00% ✓	100.00% ✓	80.00%	100.00%
	Gemini-2.5-Pro	100.00% ✓	100.00% ✓	80.00%	85.71%
	DeepSeek-R1	100.00% ✓	100.00% ✓	80.00%	100.00%
Code Translation	Claude-Sonnet-4	100.00% ✓	100.00% ✓	100.00%	100.00%
	GPT-5	100.00% ✓	100.00% ✓	100.00%	85.71%
	Gemini-2.5-Pro	100.00% ✓	100.00% ✓	100.00%	100.00%
	DeepSeek-R1	100.00% ✓	100.00% ✓	100.00%	100.00%

(b) Etcd Raft

Agent	LLM	Syntax	Runtime	Conformance	Invariant
Basic Modeling	Claude-Sonnet-4	100.00% ✓	25.00% ✓	7.69%	69.23%
	GPT-5	47.87% ✗	-	-	-
	Gemini-2.5-Pro	50.00% ✗	-	-	-
	DeepSeek-R1	50.00% ✗	-	-	-
Code Translation	Claude-Sonnet-4	100.00% ✓	66.67% ✓	15.38%	92.31%
	GPT-5	100.00% ✓	20.00% ✗	-	-
	Gemini-2.5-Pro	44.44% ✗	-	-	-
	DeepSeek-R1	100.00% ✓	0.00% ✗	-	-

Phase 3: Why?

